# Introduction To Ansible Roles

Mitesh · Follow
3 min read · Sep 27, 2018



Ansible is a handy configuration management tool as we have already looked here. It helps in configuring large number of the servers from one single controller instance. It helps automate complex tasks like configuration management, application deployment, creating CI/CD pipeline etc. Writing ansible code to manage the same service for multiple environments or different products increase code redundancy. With more complexity in functionality, it becomes difficult to manage everything in one ansible playbook file. Sharing code among teams become difficult. Ansible Role helps solve these problems. Ansible role is an independent component which allows reuse of common configuration steps. Ansible role has to be used within playbook. Ansible role is a set of tasks to configure a host to serve a certain purpose like configuring a service. Roles are defined using YAML files with a predefined directory structure.

A role directory structure contains directories: defaults, vars, tasks, files, templates, meta, handlers. Each directory must contain a main.yml file which contains relevant content. Let's look little closer to each directory.

1. **defaults**: contains default variables for the role. Variables in default have the lowest priority so they are easy to override.
2. **vars**: contains variables for the role. Variables in vars have higher priority than variables in defaults directory.
3. **tasks**: contains the main list of steps to be executed by the role.

4. **files**: contains files which we want to be copied to the remote host. We don't need to specify a path of resources stored in this directory.
5. **templates**: contains file template which supports modifications from the role. We use the Jinja2 templating language for creating templates.
6. **meta**: contains metadata of role like an author, support platforms, dependencies.
7. **handlers**: contains handlers which can be invoked by "notify" directives and are associated with service.

Let's try to understand ansible role with our nginx example. In this example, we are going to setup nginx with a config file which can read all configurations provided at a fixed directory path. We need to restart nginx when we update default configuration with our config file. For restarting nginx, we are going to use handler whose task is to listen to restart event and restart nginx once the event is received. We only need three folders to setup nginx : tasks, files, and handlers.

First, we are going to create config file "nginx.conf" which contains our nginx config to be applied on nginx.

```
user nginx;
worker_processes auto;
pid /var/run/nginx.pid;
error_log /var/log/nginx/error.log;

# Load dynamic modules. See /usr/share/doc/nginx/README.dynamic.
include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
}

http {
    log_format  main  '$remote_addr - $remote_user [$time_local]
"$request" '
                      '$status $body_bytes_sent "$http_referer" '
                      '"$http_user_agent" "$http_x_forwarded_for"';

    access_log  /var/log/nginx/access.log  main;

    sendfile            on;
    tcp_nopush          on;
    tcp_nodelay         on;
    keepalive_timeout   65;
    types_hash_max_size 2048;
```

```
    include             /etc/nginx/mime.types;
    default_type        application/octet-stream;

    # Load modular configuration files from the /etc/nginx/conf.d
directory.
    # See http://nginx.org/en/docs/ngx_core_module.html#include
    # for more information.
    include /etc/nginx/conf.d/*.conf;

    index   index.html index.htm;
}
```

In this configuration file, we are going to import all config files defined in our “/etc/nginx/conf.d/” folder with .conf extension. We are going to put all our domain level nginx config file in this path.

In handlers directory, we are going to have our main.yml file which contains code to restart nginx on receiving the event.

```
---
- name: Restart Nginx
  service: name=nginx
  state=restarted
  become: yes
```

In the final step, our main tasks directory contains code in main.yml to enable nginx on Amazon Linux 2 and then install using yum command. Once nginx is installed, we are going to copy nginx.conf file from files directory to “/etc/nginx/nginx.conf” nginx config file directory on the remote instance. Once this is copied, we are going to trigger an event “Restart Nginx” which is handled by the handler to restart nginx. This enables nginx to use this new config file.

```
- name: Enable nginx
  shell: "amazon-linux-extras enable nginx1.12"
  become: yes

- name: Install nginx
  yum:
```



```
    src: "{{ role_path }}/files/nginx.conf"
    dest: /etc/nginx/nginx.conf
```

```
    mode: 0644
  notify: Restart Nginx
```

Playbook needs to use the nginx role to setup nginx as defined below.

```
---
- hosts: all
  roles:
    - nginx
```

All this ansible code helps us setup nginx on our remote instance with our custom config file.

Nginx role code can be found here.

*PS: If you liked the article, please support it with claps. Cheers*

Ansible | Configuration Management | Nginx | Cloud

Follow

## Written by Mitesh

671 Followers

### More from Mitesh